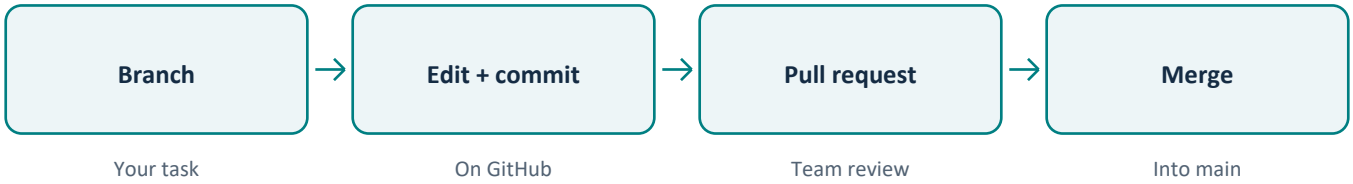
GitHub team handbook

ENGLISH / macOS + WEBSITE FIRST / VERSION 2.0 / 17 SEPTEMBER 2026

STS2026
One team.
One shared project.

Start in Safari or Chrome on your Mac. Accept your invitation, create a branch, edit a file and open a pull request on GitHub. Use a local development environment when you need to run code.



Your project

Repository: github.com/liquidtoy001/STS2026
Default teammate platform: macOS
Main interface: github.com in your browser
Optional local folder: ~/Documents/STS2026
Shared baseline branch: main

For the in-person onboarding session
Read page 5 to accept the invitation. Then complete the browser-only practice on page 19. You do not need to install Git or GitHub Desktop for those steps.

The initial main branch must exist before branch-based exercises can begin. The maintainer setup is on page 4. Examples name proposed files and branches; they do not imply those files already exist.

All diagrams are explanatory illustrations. Button names and positions may vary slightly as GitHub changes its interface.

Find the right page

Joining the team? Start on page 5. Keep page 22 open during everyday work.

03 Understand website edits and local Git

04 Maintainer: publish the initial baseline

05 Accept your invitation and open STS2026

06 Create a task branch on the website

07 Create, edit or upload files on GitHub

08 Open and review a pull request

09 Merge and start the next task

10 Bring main updates into your PR branch

11 Move between website edits and local Mac work

12 Work with notebooks and record experiments

13 Decide which project files belong in Git

14 Fix access, upload and push problems

15 Resolve a conflict on the website

16 Recover work on the wrong branch

17 Undo an edit or recover a deleted file

18 Handle large files and exposed secrets

19 Complete your first browser-only practice PR

20 Maintainer: invitations, rules and reviews

21 Official references and glossary

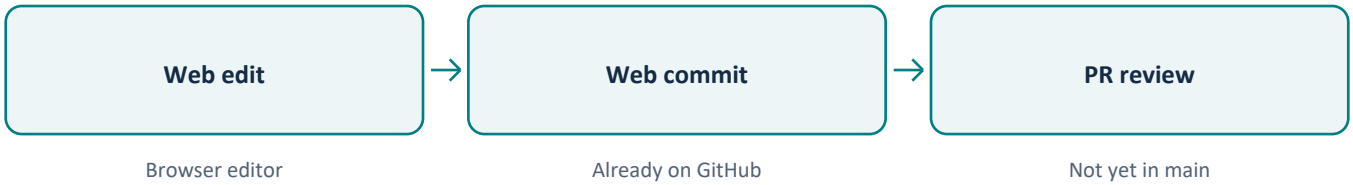
22 One-page website workflow checklist

23 Optional: set up local development on macOS

Website or local Git?

Choose the interface that fits the work; both use the same repository history.

On github.com	In a local clone on your Mac
Edit a text file or upload a small file, then click Commit changes. The commit is already on GitHub.	Edit files in an editor. Commit records them locally; push uploads the commits to GitHub.
Select a branch to see its current server-side files. No local pull is needed to view that branch on the website.	Fetch/pull to bring remote changes into your local copy. Refreshing a browser tab does not update local files.
Create PRs, review changes and discuss issues. Suitable for documentation and small edits.	Run notebooks, tests and training in an appropriate environment. Better for substantive code changes.



Term	Meaning
Branch	A named line of work. main holds reviewed work; a task branch holds your proposed changes.
Commit	A recorded snapshot with an author and message. Where you create it determines whether a separate push is needed.
Pull request / PR	A request to merge your source (compare/head) branch into the target (base) branch.
Clone / origin	Clone creates a local repository with history. origin is the usual nickname for its GitHub remote.
Download ZIP	A snapshot of files, without Git history or a configured connection for pushing changes.

Browser editing does not run your code

Viewing a notebook, saving an edit or merging a PR is not execution. github.dev is also an editor, not a Python runtime. Use the documented local or team compute environment for verification.

Publish the first baseline

MAINTAINER ONLY / Complete this once before teammates create task branches.

The local setup previously had no commits. If main is already published when you read this, skip initial creation and follow normal PR workflow. Do not initialize a second, unrelated history in the existing local project.

01. Review the source notebook and intended materials. Exclude credentials, datasets, checkpoints, temporary output and private information. Prepare a README with real setup and data-access instructions.

02. For a small new repository, use the empty repository page link to **upload an existing file**. Upload only reviewed baseline files and write an initial commit message. If GitHub requires a first file, create README.md first, then use **Add file > Upload files**.

03. Confirm the initial branch is **main**, make it the default if needed, and check the uploaded file paths. This initial main commit is the one-time setup exception.

04. Invite teammates using **Settings > Collaborators > Add people**. Confirm the matching username before sending each invitation. Page 5 explains their acceptance steps.

05. Tell everyone when the baseline is ready. After this, normal changes go through task branches and PRs.

If an existing local repository is the source
Publishing its reviewed first commit with Git is also valid. Choose one initial publication route. If you publish through the website first, clone the published repository into a new folder before integrating local work; do not force-push an independently initialized history.

Check before inviting the whole team

Open the repository as the maintainer and confirm the baseline files are present, the README explains how to run the project, and no secrets or bulk artifacts were included. Web uploads have a 25 MiB per-file limit; inspect even smaller files before sharing.

Accept your invitation

IN-PERSON ONBOARDING / All steps here happen in your browser.

01. Send your GitHub username or profile link. On your Mac, sign in at github.com. Open your avatar menu and choose **Your profile**. Share the username in github.com/YOUR_USERNAME, not only your display name.

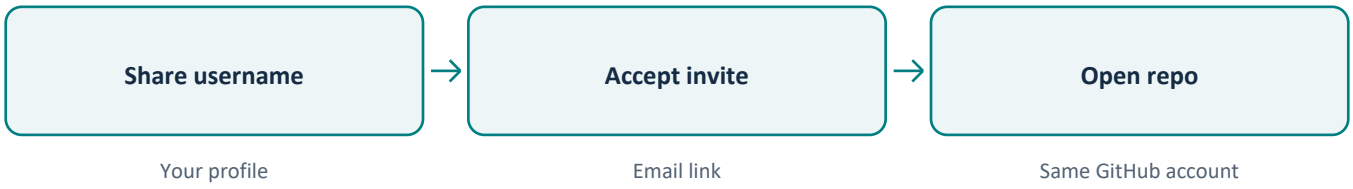
02. Wait while the maintainer invites you. Stay signed in to that same account. If you have multiple accounts, check which one is active before opening the invitation.

03. Open the invitation email. Check the inbox associated with your GitHub account, including spam/junk. Follow the GitHub invitation link for **liquidtoy001/STS2026**.

04. Click Accept invitation on GitHub. Sign in if prompted and check the owner and repository name. Wording may vary. Opening the email alone does not accept the invitation.

05. Confirm with the maintainer. Open the repository link below and say that you accepted. The maintainer should verify you are a collaborator rather than a pending invite. Viewing a public repo alone does not prove write access.

Open STS2026 - github.com/liquidtoy001/STS2026



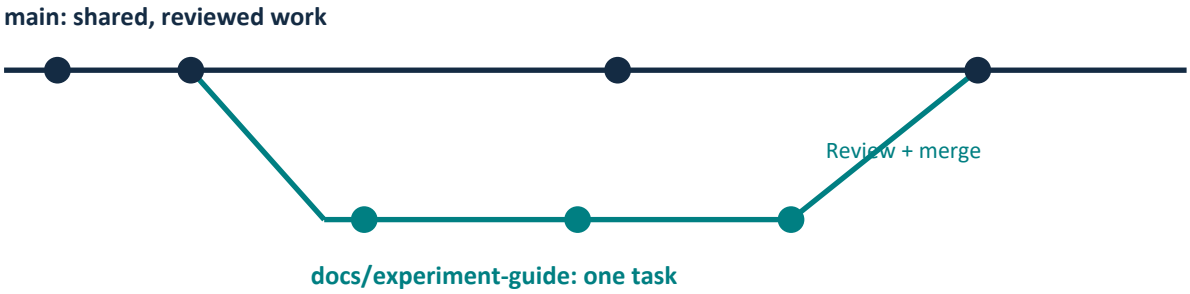
No email, invalid link or 404?
Check the username, current login and spam folder. Check for a GitHub invitation notification. Ask the maintainer to inspect the pending invite and share its invitation link if available, or resend it.

Next: your first website task

Once main contains the initial files, follow page 6 to create a branch, or page 19 for a complete practice PR. Installing GitHub Desktop and cloning the repo are optional until you need local code execution.

Create a task branch

WEBSITE / One short-lived branch for one coherent task.



01. Agree the task and owner. Our running example is adding **docs/experiment-guide.md**. Coordinate before two people edit the same notebook.

02. Open STS2026 on GitHub and select the **Code** tab. In the branch dropdown above the files or in the file tree, choose **main**.

03. Open the dropdown again. Type **docs/experiment-guide** in the branch search/create field, then choose the option to create it from **main**.

04. If the shortcut is not shown, choose **View all branches > New branch**, enter the name, set **Source: main**, and create the branch.

05. Before editing, confirm the branch selector says **docs/experiment-guide**. This branch already exists on GitHub; you do not need to publish or push it separately.

Example branch	Task
docs/experiment-guide	Explain a reproducible experiment workflow.
feat/evaluation-summary	Add one evaluation feature.
fix/audio-loading	Fix a specific input-loading problem.

No create-branch option?
Confirm your invitation was accepted and you have write access. Do not create an unexpected fork to work around a missing team permission; ask the maintainer to check.

Edit, create or upload

WEBSITE / Always check the task branch before clicking Commit changes.

Edit an existing text file

01. Select your task branch, open the file and click the pencil / **Edit this file** control. Make a focused change. Use the preview/diff view if available.

02. Click **Commit changes...**, write a specific message, and confirm the target. Commit directly to **your task branch**, not main. Complete the dialog, then inspect the saved file and commit.

Create a new document

03. On your task branch choose **Add file > Create new file**. Enter **docs/experiment-guide.md**; slashes create folders. Add text, preview the Markdown, and commit to the task branch.

Upload a small file from your Mac

04. Navigate to the intended folder on your task branch. Choose **Add file > Upload files**. Select the file from Finder, check its destination and filename, then commit. Review the resulting diff, especially if replacing a file.

A website commit is already uploaded
For the normal github.com file editor and upload page, completing Commit changes writes the commit on GitHub. There is no separate Push button. You still need a PR to merge your task branch into main.

Good commit message	Too vague
docs: explain experiment setup	update
fix: handle empty audio input	fix stuff

Browser uploads allow up to 25 MiB per file and up to 100 files per upload. These are limits, not a recommendation to upload datasets. Review notebook outputs and never upload .env, .git, environments or model weights.

Open and review a PR

WEBSITE / Turn task-branch changes into a reviewable proposal.

01. Click **Compare & pull request** after committing, or use **Pull requests > New pull request**. Select **base: main** and **compare: docs/experiment-guide**.

02. Read the proposed diff. Add a clear title and description, then click **Create pull request**. Use a draft if available for unfinished work; otherwise label it clearly as not ready to merge.

03. Request a teammate in **Reviewers** if available, or ask in the PR discussion. The reviewer opens **Files changed**, reads the diff and uses **Review changes** to comment, approve or request changes.

04. Address comments by editing the **same source branch** and committing again. The existing PR updates. Recheck the branch selector when navigating from a PR back to the file editor.

Copyable description

```
Purpose
Explain the experiment setup for new teammates.

Changes
- Describe what changed and why.

Validation
- State exactly what you checked.
- State what you did not run and why.

Known limitations
- List remaining work or missing information.
```

Review is more than clicking Approve
Check correctness, scope and validation evidence. A browser preview does not execute code. If important changes follow approval, ask for another review. The team recommends one other reviewer; enforcement depends on settings.

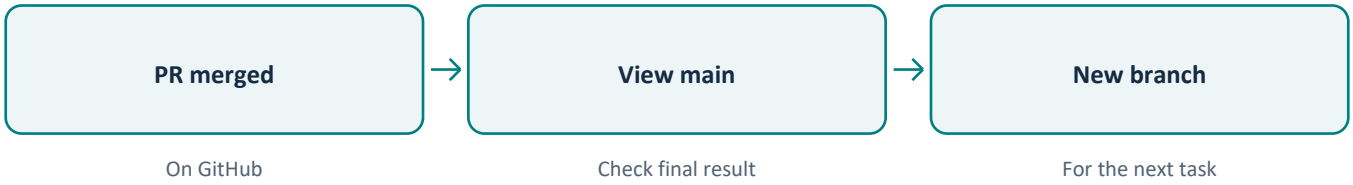
Merge and finish

WEBSITE / The maintainer coordinates initial team merges.

01. The maintainer verifies the PR targets main, review comments are addressed, and required checks have passed. For a small task, the team can standardize on **Squash and merge** if enabled.

02. After confirmation, check that the PR says **Merged**. Return to **Code**, select **main**, and verify the resulting file. Refresh the page if necessary.

03. Use **Delete branch** on the merged PR only when nobody needs that branch for more work. Start the next task on a fresh branch from main.



Website-only work has no local pull step
Once the PR is merged, selecting main on GitHub shows the server-side result. A copy of the project already downloaded to your Mac is separate and must be updated.

Optional: update an existing local clone

From the local repository, first preserve any unfinished edits and ensure the working tree is clean. Then:

```
git switch main
git pull --ff-only origin main
```

Do not reuse a merged task branch for another task. After squash merging, local git branch -d may refuse deletion because the original commits are not ancestors of main. Leave the branch until its contents are verified; do not force-delete just to remove the warning.

If the PR is closed but not merged

Closed and Merged are different states. A closed, unmerged PR did not add its changes to main. Read the discussion before restarting or reopening the work.

Update a PR with main

WEBSITE / Refreshing a page does not merge main into your task branch.

A teammate merged changes while your PR was open. Your task branch may now be behind main. Integrating those changes helps you check compatibility before your own PR is merged.

01. Open your PR and confirm **base: main** and the correct source branch. Scroll to the merge section near the bottom.

02. If GitHub offers **Update branch**, use its merge option. This brings main into your PR branch. Avoid **Update with rebase** in this beginner workflow unless the maintainer coordinates it.

03. Read the updated diff and rerun affected checks in the appropriate execution environment. The update is already on GitHub.

04. If conflicts are reported, follow page 15. If no Update branch control appears, it may not be needed or available under current settings. Ask the maintainer; do not create a reverse PR into main as a workaround.

Action	What it does
Select main in the website	Displays main; does not change the task branch.
Update branch on the task PR	Integrates base main into the source branch when available.
Merge the reviewed PR	Integrates the task result into main.
Pull in a local clone	Updates that local branch from its configured remote counterpart.

Optional local fallback - start with a clean working tree

```
git fetch origin
git switch docs/experiment-guide
git pull --ff-only origin docs/experiment-guide
git merge origin/main
# Resolve conflicts and validate, then:
git push
```

If the fast-forward-only pull fails, preserve the work and inspect the divergence with the maintainer. Do not force-push.

Website and local Mac work

Use one editing location at a time for the same branch.

If you work only on the website

Finish a small commit before switching browser tabs, branches or computers. Uncommitted editor text is not a shared Git commit; copy valuable drafts elsewhere before navigating away. Reopen the branch on another device to see committed work.

Continue a website branch on your Mac

Set up a clone using page 23. From a clean working tree, fetch and create a local tracking branch for the published branch. The tracking command below is for a branch that does not already exist locally.

```
cd ~/Documents/STS2026
git fetch origin
git switch --track origin/docs/experiment-guide
```

If it already exists locally, use git switch docs/experiment-guide and git pull --ff-only instead. Run the documented project checks, then commit and push your intended changes.

Return to editing on the website

Push local commits first and verify them on GitHub. After later website commits, pull that same branch before resuming local edits. Browser commits and local commits can diverge if both locations change independently.

```
git status
git add -- docs/experiment-guide.md
git diff --cached
git commit -m "docs: clarify experiment setup"
git push
```

Pause unfinished local work
Either make a clearly labelled checkpoint commit on your task branch, or use a local stash with help. git stash push -u includes untracked files but not ignored files. A stash is local and is not a GitHub backup. Preserve it until restored changes are checked.

In macOS Terminal, ~ means your home directory. Teammates may choose another local folder; never use the maintainer's Windows drive path on your Mac.

Notebooks and experiments

WEBSITE FOR REVIEW / A real execution environment for running code.

The project includes vits-light_v1.ipynb. GitHub can display notebook content, but its repository page does not run notebook cells. Editing notebook JSON in a text editor is fragile; use a notebook-aware local or team environment.

01. Agree a primary editor for each notebook task. Coordinate before anyone else modifies the same notebook; whole-file uploads can overwrite another person's changes.

02. Start from the latest intended branch. Run a small documented check before changing code. For substantive notebook work, prefer a local clone so you can inspect and commit the actual diff.

03. Before committing, inspect cell outputs, metadata and paths. Remove disposable bulk output and secrets while preserving the evidence needed for the report.

04. Restart the kernel and execute the relevant cells in order where practical. In the PR, distinguish a small sample, import check and full experiment. Do not mark an unrun change as tested.

Record	Purpose
Code commit + local edits, if any	Identify the code that actually ran.
Configuration and random seed	Make comparisons repeatable.
Data version and split	Keep evaluation comparable.
Environment and compute	Explain dependency and runtime differences.
Metrics and artifact location	Connect reported results to evidence.

Mac hardware is not automatically the training environment
A project that needs NVIDIA CUDA cannot use a Mac's Apple GPU as a drop-in replacement. Follow the project's documented CPU, supported Mac or remote GPU setup. Do not promise training support until it is verified.

For notebook conflicts, preserve both versions, agree a base and reapply intended cell changes with the other editor. Verify the resolved notebook opens and runs. Do not blindly accept an entire side.

What belongs in Git?

Review uploaded files as carefully as locally committed files.

Track and review	Keep outside normal Git history
Source, README, dependency definitions	Virtual environments, caches and scratch files
Placeholder-only configuration examples	Real .env files, passwords and tokens
Small curated metrics and report figures	Datasets, checkpoints and bulk training outputs
Data access and version instructions	Private material and machine-specific paths

Suggested .gitignore starting point

```
.DS_Store
.env
.venv/
venv/
__pycache__/
.ipynb_checkpoints/
tmp/
data/
datasets/
checkpoints/
*.pt
*.pth
*.ckpt
```

Web uploads still need manual review
.gitignore guides local Git file selection; do not rely on it to prevent manually selected website uploads. It also does not remove already tracked files or erase previous commits.

Do not ignore all of output/ automatically. A selected report figure or this PDF may belong in Git; large temporary artifacts should have a separate location. On macOS, .DS_Store stores Finder metadata and should normally be excluded.

Share datasets and checkpoints through team-agreed storage with access instructions and versions. Git LFS requires deliberate configuration and storage planning; adding it later does not automatically clean old Git history.

Access and upload problems

WEBSITE FIRST / Identify the account, repository and branch before retrying.

Symptom	What to do
Repository not found / 404	Check the exact URL, active account and accepted invitation. A private repo can look missing if you lack access.
Cannot create a branch or edit	Have the maintainer confirm collaborator access and branch rules. Watching or starring the repo does not grant write access.
GitHub proposes a fork	For this shared-repo workflow, check that you are signed in to the invited account with write access.
Upload fails or file is too large	Browser uploads allow 25 MiB per file and 100 files per batch. Do not split a model into chunks to bypass the project's storage rules.
Cannot commit a stale web edit	Copy your draft first. Reload the current branch version and reconcile changes; do not overwrite another person's newer work.
PR cannot merge	Read whether it needs approval, checks, branch updates or conflict resolution. Page 15 covers conflicts.
Empty repo / no main branch	The maintainer must publish the baseline; see page 4.

Optional local Git errors

For authentication failure, reauthenticate the affected client; a browser login does not automatically configure Terminal. For non-fast-forward / fetch first, preserve work, fetch and inspect divergence. Do not force-push.

Ask for useful help

Share the URL, branch, action and exact error, with secrets removed. Say whether you edited in the browser or locally. For local issues include git status. This tells the maintainer which copy needs attention.

Resolve a web conflict

WEBSITE / Use this for simple text conflicts, not notebook JSON or binary files.

01. Open the task PR. Read the conflict warning and click **Resolve conflicts** if enabled. Confirm the base is main and the head is your task branch.

02. Read both versions and agree the intended result with the other author. Use the branch labels beside each conflict to identify the sides; do not infer ownership from top/bottom position alone.

```
<<<<<<< docs/experiment-guide
Record the sample list.
=======
Record the random seed.
>>>>>>> main
```

03. Edit one correct combined result and remove the conflict markers. Check all conflicts in every listed file. Click **Mark as resolved** for each completed file.

04. Click **Commit merge** and confirm the intended head branch. This merges the entire base branch into the task branch; it does not finish merging the task PR into main.

05. Inspect the updated diff and run the relevant checks in the execution environment. Ask for review again if needed; the maintainer can then merge the PR.

Resolve conflicts is disabled?
Some conflicts require a local Git client. Preserve your work and ask the maintainer to help. For notebooks, preserve both versions and resolve cell changes in a notebook-aware environment. For PDFs/images, agree a version or regenerate from source.

If a local merge must be abandoned

git merge --abort applies to an in-progress local merge, not a website commit that already exists. It can discard resolution edits; use it only with a understood clean pre-merge state and preserve anything valuable first.

I used the wrong branch

A completed website commit is already shared on GitHub.

State	Recovery
Still editing in the browser; not committed	Copy your draft somewhere safe. Cancel/navigate back, select or create the correct task branch, reopen the current file and reapply the intended edits.
Website commit landed on main	Tell the maintainer immediately. Decide whether it can stay or needs a corrective/revert PR. There is no unpublished local commit to undo.
Website commit landed on another task branch	Record the commit link and notify that branch owner. Coordinate moving the intended change and correcting the original branch without losing their work.
Local uncommitted edits on main	Create a rescue/task branch before committing. Review which edits came with you.
Local main commits not pushed	Create a rescue branch to preserve the commits. Have the maintainer inspect history before restoring main.

Local-only first step, when preserving a local commit or edit

```
git status
git switch -c fix/rescue-work
```

This creates a branch name for your current state; it does not reset main. Review and commit uncommitted edits on the rescue branch if needed. Do not use this as a remedy for an already published web commit.

Prevent the next mistake
Check the branch selector before opening the editor and check the commit destination again in the Commit changes dialog. Multiple browser tabs may point to different branches.

Undo or recover a file

WEBSITE / Prefer an explicit corrective change that preserves shared history.

A small incorrect website edit

01. Record the bad commit or PR link. Create a fresh correction branch from main if the change is already merged, or correct the existing task branch if its PR is still open.

02. Open the file history to inspect the last correct version. Reapply only the needed correction to the current file; replacing it wholesale may erase later valid changes.

03. Commit the correction, explain it in the PR and request review. The earlier bad commit remains in history; the correction records what changed.

Revert a merged PR

For a merged PR, the maintainer can use **Revert** if GitHub offers it. GitHub prepares a new revert PR; inspect, validate and review it before merging. If the button is unavailable or conflicts occur, coordinate a local revert. Do not assume every merge type has the same recovery steps.

Restore a deleted file

Find the last intended version through file/commit history, then recreate it at the correct path on a recovery branch and open a PR. Review whether other code depends on its deletion. A never-committed local file cannot be recovered from Git history.

Local-only undo is a different operation
GitHub Desktop can undo a latest unpushed local commit. That does not apply to github.com commits, which are already remote. Discarding uncommitted edits can lose work that Git never recorded.

Before copying old content

Keep the current version, identify the exact change you want to reverse, and inspect later edits. Never use force push or hard reset as a routine way to undo a shared mistake.

Wrong files or secrets

Deleting a file in a later commit does not erase its earlier contents.

An unintended upload

01. Before committing: remove the file from the upload selection or cancel and restart with the correct selection. Check every filename and destination.

02. After a website commit: it is already in remote history. Tell the maintainer. A deletion PR stops the file appearing in the latest snapshot, but not in previous commits.

03. Large files or older local commits: ask the maintainer to inspect the history before retrying. A fresh deletion does not fix a large blob already inside an earlier commit.

If a credential was uploaded

04. Revoke or rotate the credential with its provider first. Treat it as exposed even in a private repository. Deleting the file alone cannot invalidate the credential.

05. Tell the maintainer the credential type and location without reposting its value. Check notebook outputs, logs and PR diffs too. Coordinate history cleanup and updates to existing clones if required.

Repository visibility is not cleanup
Making a repository private or deleting a branch does not revoke a credential or guarantee all copies are removed. Follow the official sensitive-data guidance with the maintainer.

Prevent repeats

Use placeholder-only examples, keep real configuration outside Git and inspect selected upload files. Avoid uploading whole Finder folders without review. For local-only tracked files, stopping tracking changes future snapshots; it still does not purge history.

Your first browser-only PR

15-20 MINUTES / No Terminal or GitHub Desktop required.

01. Accept the invite using page 5. Open STS2026 in Safari or Chrome and select main after the maintainer publishes the baseline.

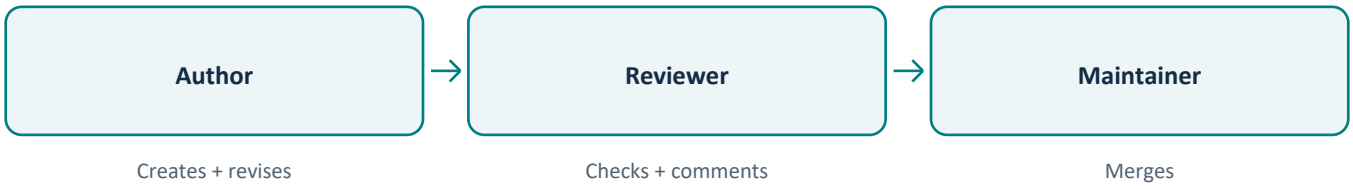
02. Create **docs/onboarding-YOUR_USERNAME** from main. Replace YOUR_USERNAME with your GitHub username.

03. On that branch choose **Add file > Create new file**. Name it **docs/team/YOUR_USERNAME.md**. Add your preferred name, intended project role and one setup question. Avoid private contact details.

04. Preview the text and click **Commit changes**. Confirm the commit goes to your task branch. Then open a PR with **base: main**.

05. Have a teammate request one useful clarification in review. Edit the same file on the same task branch and commit again. Confirm the existing PR updates.

06. After approval, the maintainer merges. Select main on the repository page and confirm your note appears. Create a new branch for your first real task.



Finished means
The PR says Merged, main contains your note, and you can explain why a website commit needs a PR but no separate push. Report any unclear step so the handbook can improve.

The exercise verifies your collaboration workflow, not the model or experiment pipeline. Actual execution checks belong in the project setup process.

Maintainer playbook

Recommended team policy; repository settings must be configured separately.

Invite each teammate in person

Collect the GitHub username/profile link. Open **Settings > Collaborators > Add people**, check the matching account, and send the invitation. Have the person complete page 5. Confirm each invite changes from pending to accepted.

Use a small, clear workflow

Keep main plus short-lived task branches. Require a PR and one other reviewer by team agreement. Assign one owner per task and one primary editor per notebook. Initially let the maintainer coordinate merges.

Enforce rules where supported

Confirm main is the default branch. Where the plan supports it, require PRs, approval and resolved conversations; disallow force pushes and deletion of main. Check bypass permissions. Private-repository branch protection depends on the account/organization plan.

Conventions and enforcement differ
If the current plan cannot enforce a rule, keep it explicit and coordinate merges manually. This handbook does not certify which protections are enabled.

Document and test real requirements

Item	Minimum useful content
README.md	Project goal, verified macOS/remote setup, dependencies and data access.
CONTRIBUTING.md	Branch, PR and review rules; links to this handbook.
Configuration example	Documented placeholder values only.
Validation	Small meaningful checks; actual experiment evidence and limitations.

Do not require a nonexistent automated check or claim a smoke test validates full training. At a report milestone, record the reviewed code commit and corresponding experiment artifacts.

References and glossary

Official GitHub documentation / checked 17 September 2026.

1. Accepting a collaborator invitation
2. Creating branches on GitHub
3. Editing files on the website
4. Uploading files and size limits
5. Updating a PR with its base branch
6. Resolving conflicts on GitHub
7. Reverting a pull request
8. Removing sensitive data
9. The github.dev editor and its limitations
10. Branch protection and plan availability

Small glossary

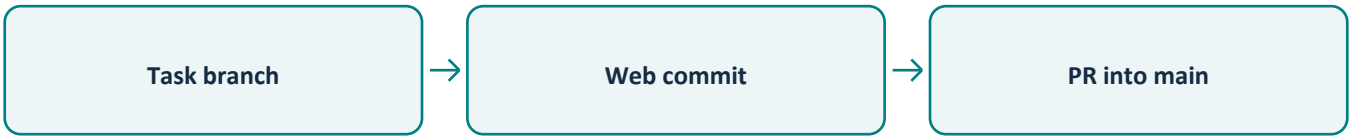
Term	Meaning
Base / head	PR destination / source branch, usually main / your task branch.
Working tree / staging area	Local checked-out files / changes selected for the next local commit.
HEAD / origin/main	Current checked-out commit / local record of remote main as of the last fetch.
Push / pull	Upload local commits / integrate remote changes into a local branch.
github.dev / Codespaces	Browser editor / separate cloud execution environment. They are not interchangeable.

Project: github.com/liquidtoy001/STS2026. Site labels may change. Example paths and validation steps must be adapted to the actual project.

Website desk reference

OPEN / BRANCH / EDIT / COMMIT / PR / REVIEW / MERGE

Moment	Check
Join	Correct GitHub account? Invitation accepted? Initial main published?
Start	Select main. Create a fresh task branch. Confirm the branch name.
Edit	Use the pencil or Add file. Check the path and preview the change.
Commit	Specific message? Correct task branch? No secrets or bulk uploads?
PR	Base main, correct source branch, clear purpose, honest validation, reviewer.
Finish	PR says Merged. Select main and verify. Start a fresh branch next time.



Three common misunderstandings

No push after a normal website commit. It is already on GitHub.
Refreshing is not branch integration. Use the PR update workflow on page 10.
Viewing is not running. Execute code in the documented environment, then report what actually ran.

Where to look when stuck

Invitation: page 5. Missing edit/upload access: page 14. Conflict: page 15. Wrong branch: page 16. Undo/recovery: page 17. Secrets/large files: page 18. Local Mac setup: page 23.

When asking for help
Send the page/PR link, branch name, intended action and exact error. Say whether you were using github.com, github.dev or a local clone. Remove passwords and tokens from screenshots.

Optional: local Mac setup

Only needed when your task requires local editing, execution or Git recovery.

Option A: GitHub Desktop for macOS

01. Install GitHub Desktop for your Mac and sign in to the invited account. Configure your author name and GitHub-associated email (or GitHub private commit email) in its Git settings.

02. Choose **File > Clone Repository > URL** and paste the STS2026 URL below. Select a new/empty destination such as **~/Documents/STS2026**. Open it in your editor.

```
https://github.com/liquidtoy001/STS2026.git
```

Option B: Terminal, if Git is installed

Open Terminal with Spotlight (Command-Space, then type Terminal). Check git --version; if macOS prompts for Command Line Tools, complete that installation first. Then, from a parent folder:

```
cd ~/Documents
git clone https://github.com/liquidtoy001/STS2026.git
cd STS2026
git remote -v
git status
```

For a private repository, HTTPS cloning needs working Git authentication. Your GitHub account password is not accepted as a Git HTTPS password. Use Desktop's sign-in flow, or a documented credential-manager/PAT or SSH setup. Never paste a token into a shared command or repository URL.

If ~/Documents/STS2026 already contains work
Do not clone over it. Choose another empty destination and preserve any existing edits. Download ZIP is fine for reading a snapshot but does not create a Git-connected working copy.

Before running the project

Follow the actual README for Python, dependencies, data and compute. Cloning does not install them. Apple Silicon, Intel Macs and GPU servers may need different environments. See page 12 for notebook validation, and page 11 for switching between local and web editing.